

Universidade Federal
de Campina Grande



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
DEPARTAMENTO DE ENGENHARIA ELÉTRICA E INFORMÁTICA
PROGRAMA DE RESIDÊNCIA EM MICROELETRÔNICA
MCTI/Softex/IRede

RELATÓRIO TÉCNICO

Semana 3 - Microarquitetura e transformações de rodada

Aluno: Guilherme José Mendes Jerônimo
Professor: Prof. Henrique Cunha
Semestre: 2026.2

Campina Grande - PB
2026

Sumário

1 Visão Geral e Fronteiras de Projeto **2**

2 Diagrama de Blocos Interno do AES Engine **2**

3 Decisões de Microarquitetura do AES **3**

3.1 Arquitetura do Datapath: Iterativa de 128 bits 3

3.2 Tabela Comparativa de Alternativas 4

4 Máquina de Estados Finita (FSM) do `minisoc_aes_top` **4**

4.1 Descrição Detalhada dos Estados 4

5 Estratégia de Clock Domain Crossing (CDC) **5**

5.1 Protocolo de Sincronização por Toggle 6

6 Sequência e Sincronização de Reset **6**

7 Módulos de Transformação de Rodada **6**

7.1 Aritmética em $GF(2^8)$ 7

7.2 Sanity Check de Inversas 7

8 Plano de Testes e Aceite da Semana 3 **7**

Resumo

Este documento apresenta a especificação detalhada da microarquitetura interna do bloco `minisoc_aes` para o sistema MiniSoC AES. São contempladas as decisões de projeto para o *datapath* iterativo de 128 bits, a máquina de estados finita (FSM) do controlador de topo e acesso à memória, as estratégias de travessia de domínio de clock (CDC) entre os domínios `spi_sclk` e `clk`, a sincronização de reset vinculada à estabilização do PLL (`PLL_LOCKED`), e a organização das transformações de rodada diretas e inversas segundo a especificação NIST FIPS-197.

1 Visão Geral e Fronteiras de Projeto

O objetivo deste documento é detalhar a microarquitetura interna do módulo `minisoc_aes` e definir as estratégias de controle, sincronização de clock (CDC) e gerenciamento de reset para o projeto MiniSoC AES.

O *top-level* do sistema (`minisoc_top`) e as interfaces externas de controle seguem estritamente a especificação do *Block Guide MiniSoC AES v1.0.0*. Este documento fixa o funcionamento interno dos seguintes componentes sob responsabilidade do aluno na Semana 3:

- `minisoc_aes_top`: Controlador FSM de alto nível do AES e gerenciador de acesso em rajada à memória compartilhada.
- `minisoc_aes_dp`: *Datapath* de criptografia e decriptação com suporte a blocos de 128 bits.
- **Transformações de Rodada Diretas e Inversas**: Implementação combinacional pura dos módulos `SubBytes`, `ShiftRows`, `MixColumns`, `AddRoundKey` e suas inversas correspondentes.
- **Mecanismos de CDC e Reset**: Sincronizador por *toggle* para a interface SPI e *Reset Synchronizer* para o domínio de clock do sistema (`sys_clk`).

2 Diagrama de Blocos Interno do AES Engine

A organização interna do bloco `minisoc_aes_top` e suas conexões com o banco de registradores do SPI e a memória compartilhada de 1024×32 bits é estruturada conforme ilustrado a seguir:

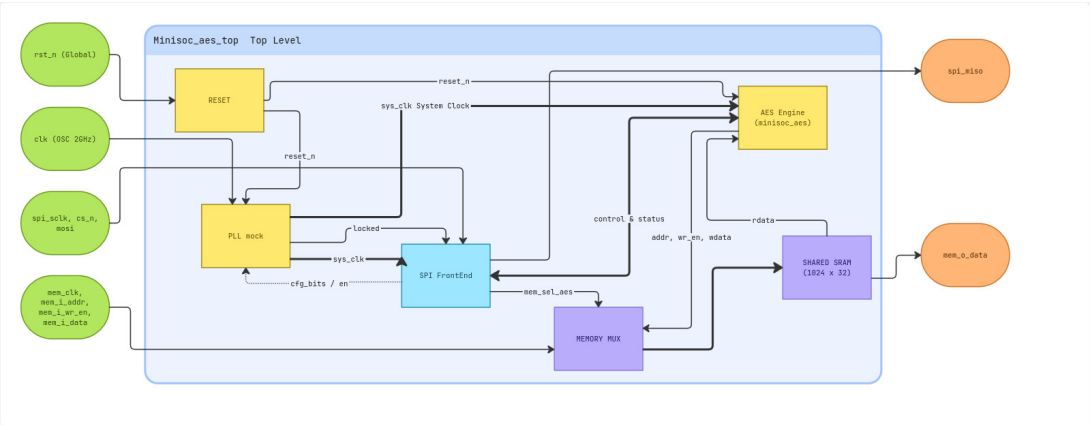


Figura 1: Diagrama de Blocos Interno do `minisoc_aes_top`.

Faz-se necessária a ampliação do funcionamento do AES Engine no seguinte diagrama:

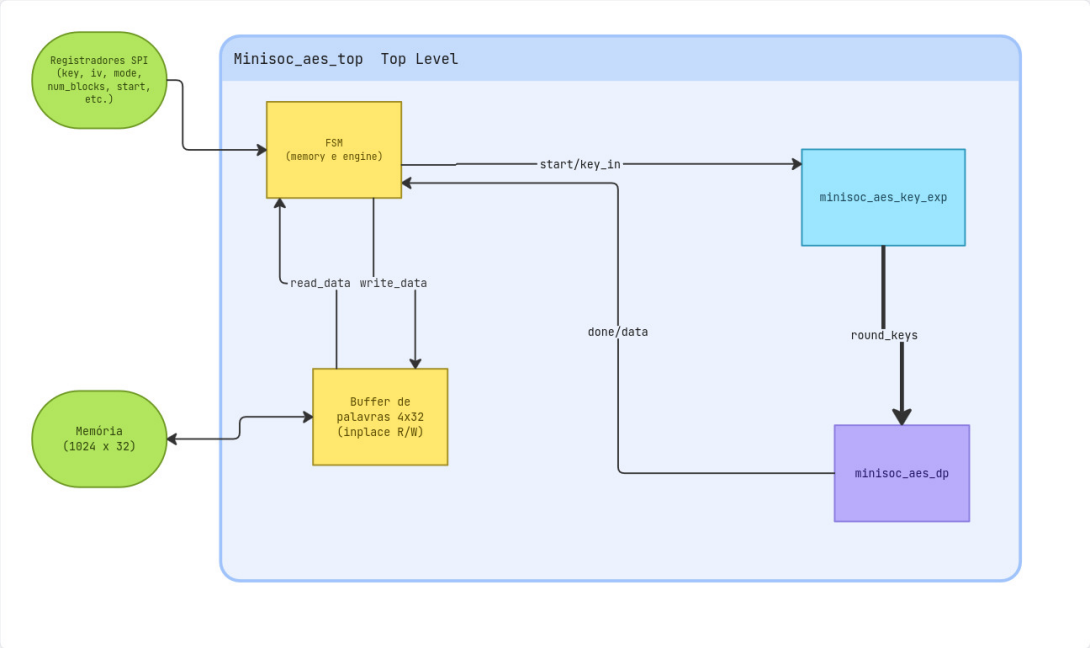


Figura 2: Detalhamento do motor criptográfico do `minisoc_aes_top`.

3 Decisões de Microarquitetura do AES

3.1 Arquitetura do Datapath: Iterativa de 128 bits

- **Escolha Adotada**: Arquitetura **Iterativa de 128 bits com 1 rodada por ciclo de clock**.
- **Justificativa**: O *datapath* processa todos os 128 bits do estado (*State*) em paralelo em cada ciclo de clock. A execução de um bloco de 128 bits consome exatamente 10 ciclos de processamento (rodadas 1 a 10). Esta abordagem otimiza a área de silício ao reutilizar os mesmos blocos combinacionais de transformação entre todas as rodadas do algoritmo.

3.2 Tabela Comparativa de Alternativas

Tabela 1: Comparação de alternativas de microarquitetura do *datapath* AES.

Métrica	**Iterativa (128-bit) [Escolhida]**	**Pipelined (10 Estágios)**	**Byte-Serial (8-bit)**
Latência por Bloco	10 ciclos	10 ciclos	$\approx$ 160 ciclos
Throughput Máximo	1 bloco / 10 ciclos	1 bloco / 1 ciclo	1 bloco / 160 ciclos
Área Relativa (Gates)	1,0× (Média)	$\approx$ 9,5× (Muito Alta)	$\approx$ 0,3× (Muito Baixa)
Complexidade da FSM	Moderada	Alta (Controle de Bolhas)	Complexa (Multiplexação)

4 Máquina de Estados Finita (FSM) do `minisoc_aes_top`

O módulo `minisoc_aes_top` assume o controle da memória compartilhada quando o registrador `SYSTEM_CONTROL.MEM_SEL_AES` é configurado em `1`. A FSM realiza a leitura sequencial de 4 palavras de 32 bits, aciona o *datapath* AES e escreve o resultado de volta na mesma região de memória (*in-place processing*).

4.1 Descrição Detalhada dos Estados

1. `IDLE`: Aguarda a escrita de `START = 1` no registrador `AES_CONTROL`. Valida se `NUM_BLOCKS` está entre 1 e 256 e se `MODE` está na faixa `000` a `101`. Caso ocorra erro, sinaliza `AES_STATUS.ERROR = 1`.

2. `FETCH_W0` a `FETCH_W3`: Lê sequencialmente quatro palavras de 32 bits dos endereços `base + 0` até `base + 3` da memória compartilhada. Recompõe o vetor de estado de 128 bits:
$$\text{Block}[127:0] = \{\text{Word}_0, \text{Word}_1, \text{Word}_2, \text{Word}_3\}$$

3. `PROCESS_BLOCK`: Sinaliza `start` para o `minisoc_aes_dp` e aguarda a conclusão informada por `dp_done`.

4. `WRITE_W0` a `WRITE_W3`: Grava as quatro palavras de 32 bits do bloco processado nos mesmos endereços (*in-place*).

5. `NEXT_BLOCK`: Incrementa o ponteiro base de memória (`base <= base + 4`) e atualiza o registrador `BLOCKS_DONE`. Quando `BLOCKS_DONE == NUM_BLOCKS`, encerra e sinaliza `DONE = 1`.

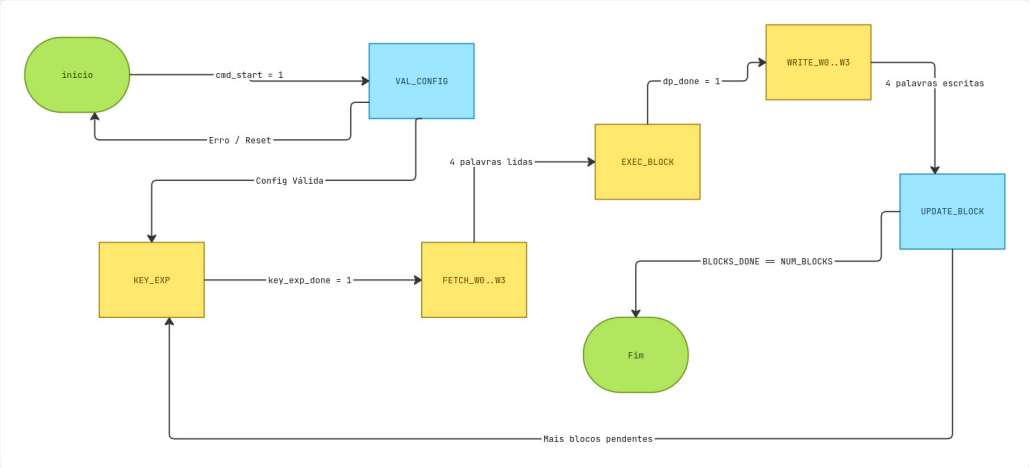


Figura 3: Diagrama da Máquina de Estados Finita (FSM) do `minisoc_aes_top`.

5 Estratégia de Clock Domain Crossing (CDC)

Conforme a Seção 5.3 do *Block Guide*, o sistema possui domínios de clock assimétricos:

- `spi_sclk`: Clock serial externo síncrono ($\leq$ 25 MHz).
- `clk`: Clock de referência *always-on* (2 GHz).
- `sys_clk`: Clock gerado pelo PLL para o engine AES (250 MHz a 2 GHz).

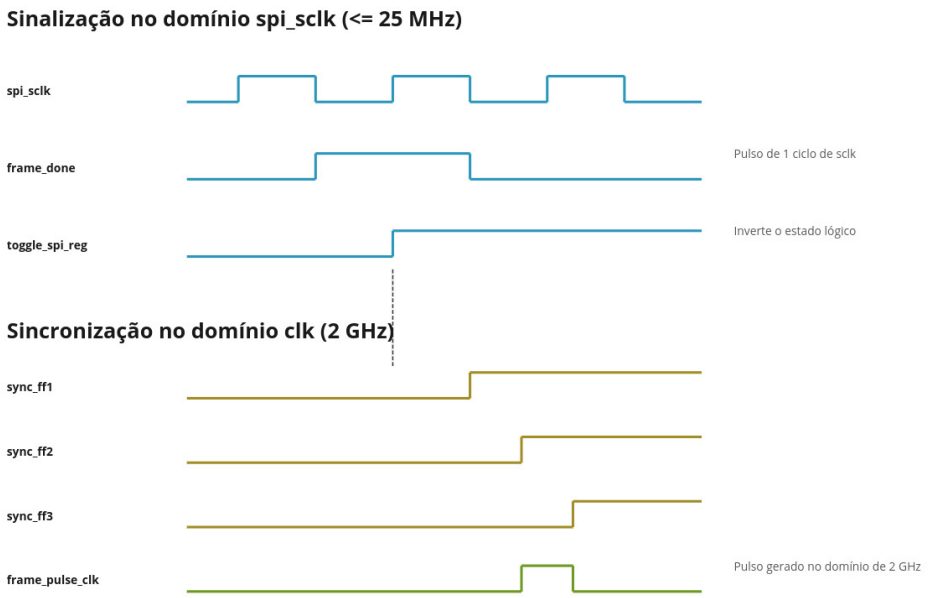


Figura 4: Esquema e temporização de Clock Domain Crossing (CDC) por *toggle*.

5.1 Protocolo de Sincronização por Toggle

1. Ao completar um quadro SPI de 40 bits no domínio `spi_sclk`, um registrador de *toggle* inverte seu estado (`toggle_spi <= ~toggle_spi`).
2. O sinal atravessa para o domínio `clk` através de um sincronizador de 2 estágios (*2-FF Synchronizer*).
3. Um detector de borda no domínio `clk` detecta a alteração de nível lógico e gera um pulso de *enable* para carregar com segurança o dado recebido nos registradores do sistema.

6 Sequência e Sincronização de Reset

Conforme a Seção 4.2 do *Block Guide*, o reset do motor AES depende da estabilização do PLL.

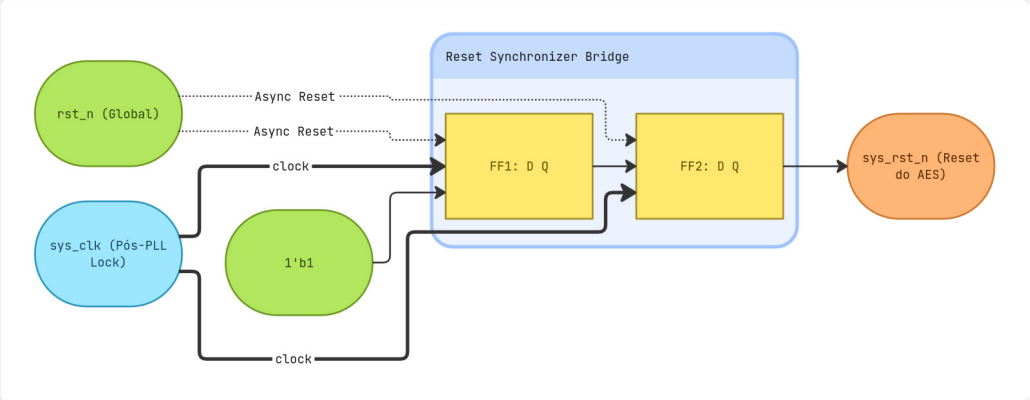


Figura 5: Estrutura do *Reset Synchronizer* e forma de onda da sequência de inicialização.

- O reset global `rst_n` inicializa a lógica de controle no domínio `clk`.
- O clock do sistema (`sys_clk`) é mantido inativo enquanto `SYSTEM_STATUS.PLL_LOCKED = 0`.
- A liberação do reset no domínio `sys_clk` (`sys_rst_n`) é realizada via **Reset Synchronizer** alimentado pelo próprio `sys_clk`, eliminando riscos de violação de *recovery/removal*.

7 Módulos de Transformação de Rodada

As transformações de rodada do AES-128 foram estruturadas em submódulos combinacionais e puros:

- **Cifragem (Diretas)**: `minisoc_aes_sub_bytes`, `minisoc_aes_shift_rows`, `minisoc_aes_mi` `minisoc_aes_add_round_key`.

- **Decriptação (Inversas)**: `minisoc_aes_inv_sub_bytes`, `minisoc_aes_inv_shift_rows`, `minisoc_aes_inv_mix_columns`.

7.1 Aritmética em $GF(2^8)$

As multiplicações matriciais de `MixColumns` e `InvMixColumns` ocorrem no Corpo de Galois $GF(2^8)$ sob o polinômio irredutível:

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (\texttt{0x11B})$$

7.2 Sanity Check de Inversas

A validação unitária garante a propriedade de identidade bijetora:

$$\text{InvSubBytes}(\text{SubBytes}(X)) = X$$

$$\text{InvShiftRows}(\text{ShiftRows}(X)) = X$$

$$\text{InvMixColumns}(\text{MixColumns}(X)) = X$$

8 Plano de Testes e Aceite da Semana 3

A validação das entregas da Semana 3 será conduzida pelo *testbench* autoverificável `tb/tb_semana_03.sv`:

1. **Vetores de Referência FIPS-197 (Apêndice B)**:
   - Entrada (*start of round*): `193de3bea0f4e22b9ac68d2ae9f84808`
   - Saída esperada em `SubBytes`: `d42711aee0bf98f1b8b45de51e415230`
   - Saída esperada em `ShiftRows`: `d4bf5d30e0b452aeb84111f11e2798e5`
2. **Teste de Inversas**: Aplicação combinada das funções inversas sobre os resultados intermediários, verificando a recuperação exata da entrada original.
3. **Análise de Lint**: Verificação estática sem nenhum alerta ou erro em ferramentas como Verilator.